

AI ASSISTANT CODING

Lab Assignment-2

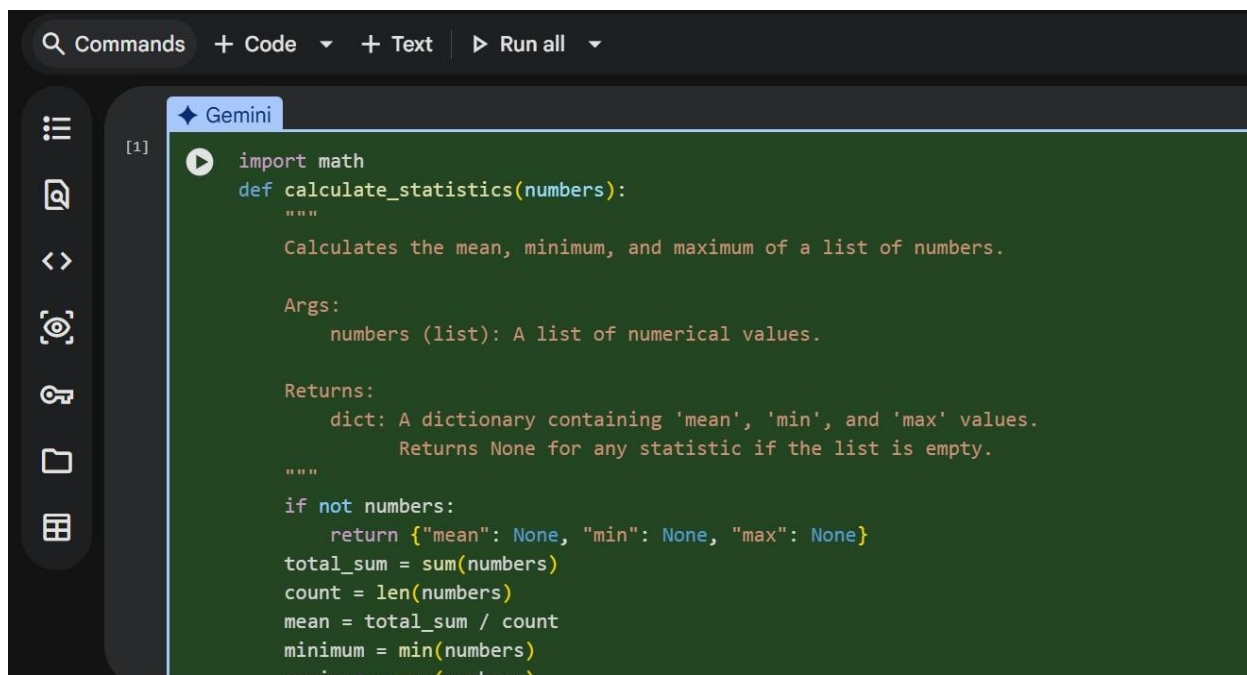
2303A54012
A.Sanjay B47A

Task 1 — Statistical Summary using Google Gemini

(Colab) Create a Python function that finds:

- Mean
- Minimum
- Maximum

from a list of numbers using **Gemini in Colab**.



The screenshot shows a Google Colab notebook interface. At the top, there is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. Below this, the notebook is titled 'Gemini'. The code cell contains a Python function definition for 'calculate_statistics' which takes a list of numbers and returns a dictionary with mean, minimum, and maximum values. The function includes docstrings and handles the case where the list is empty.

```
[1] import math
def calculate_statistics(numbers):
    """
    Calculates the mean, minimum, and maximum of a list of numbers.

    Args:
        numbers (list): A list of numerical values.

    Returns:
        dict: A dictionary containing 'mean', 'min', and 'max' values.
        Returns None for any statistic if the list is empty.
    """
    if not numbers:
        return {"mean": None, "min": None, "max": None}
    total_sum = sum(numbers)
    count = len(numbers)
    mean = total_sum / count
    minimum = min(numbers)
    maximum = max(numbers)
```

```
if not numbers:
    return {"mean": None, "min": None, "max": None}
total_sum = sum(numbers)
count = len(numbers)
mean = total_sum / count
minimum = min(numbers)
maximum = max(numbers)

return {"mean": mean, "min": minimum, "max": maximum}

# Example usage:
my_list = [10, 20, 5, 15, 30, 25]
stats = calculate_statistics(my_list)
print(f"For the list {my_list}:")
print(f"  Mean: {stats['mean']}")
print(f"  Minimum: {stats['min']}")
print(f"  Maximum: {stats['max']}")

empty_list = []
empty_stats = calculate_statistics(empty_list)
print(f"\nFor an empty list:")
```

Terminal

Commands + Code + Text Run all

```
[1] print(f"  Statistics: {empty_stats}")

single_number_list = [42]
single_stats = calculate_statistics(single_number_list)
print(f"\nFor a list with a single number {single_number_list}:")
print(f"  Mean: {single_stats['mean']}")
print(f"  Minimum: {single_stats['min']}")
print(f"  Maximum: {single_stats['max']}")
```

... For the list [10, 20, 5, 15, 30, 25]:
Mean: 17.5
Minimum: 5
Maximum: 30

For an empty list:
Statistics: {'mean': None, 'min': None, 'max': None}

For a list with a single number [42]:
Mean: 42.0
Minimum: 42
Maximum: 42

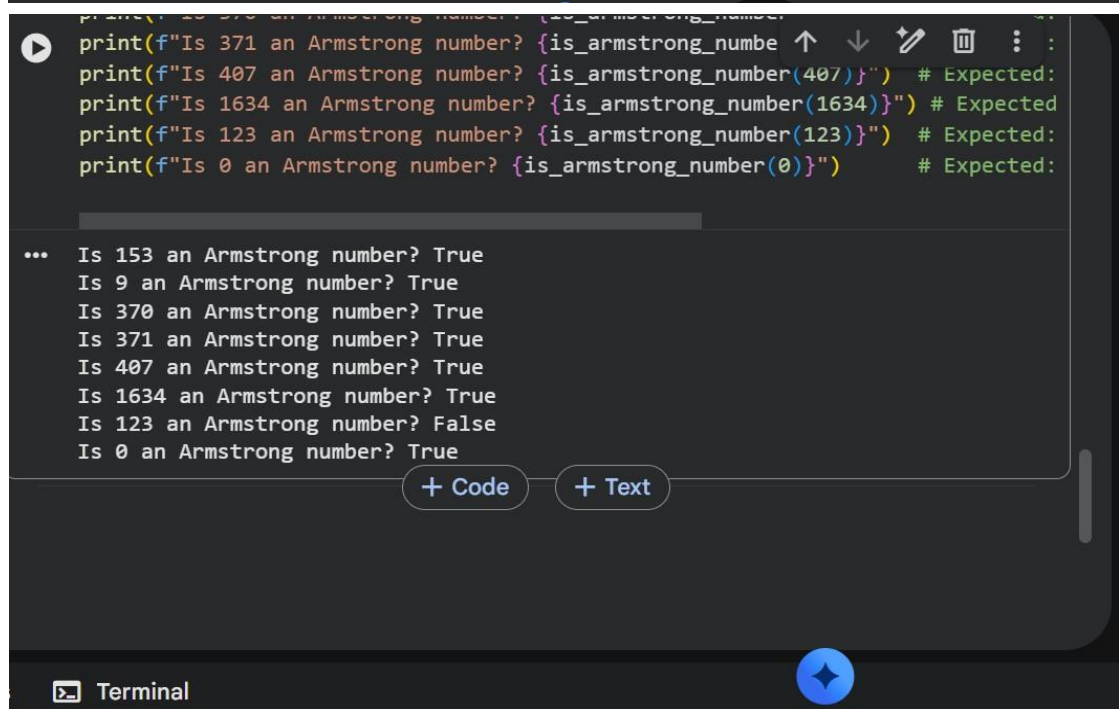
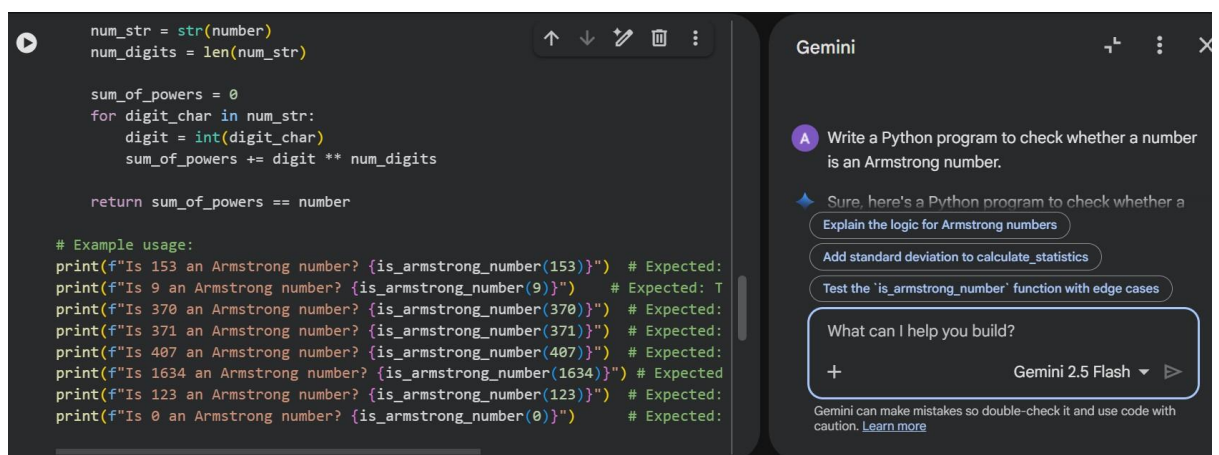
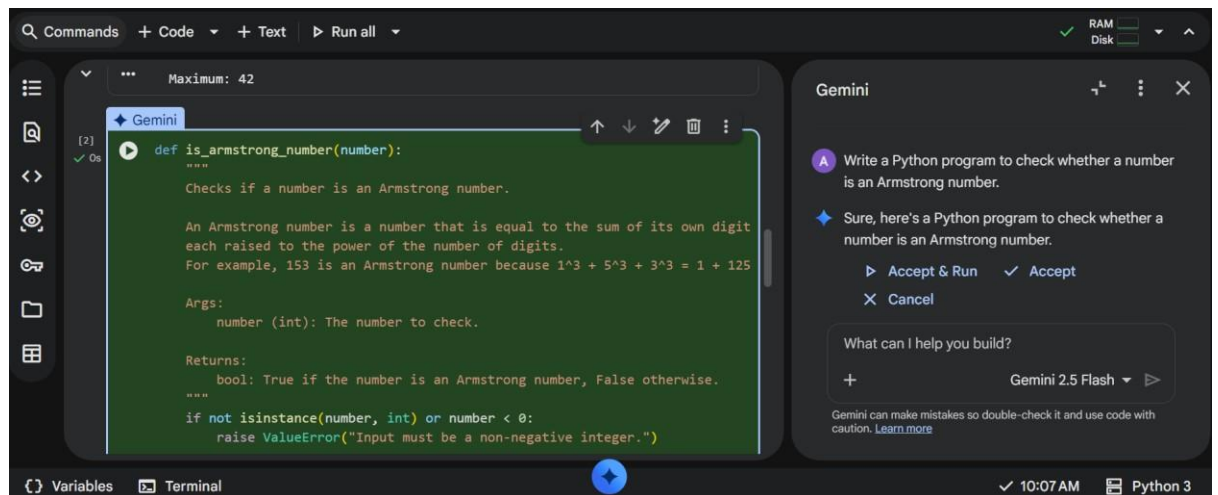
Variables Terminal

Task 2 — Armstrong Number (Gemini vs Copilot) Generate

Armstrong number checker using:

- Gemini

- GitHub Copilot ,Then compare them.



Feature	Gemini	Copilot
Logic	Uses step-by-step digit extraction and power calculation, easy to follow	Uses the same core logic but often written in a more compact style
Code Clarity	Well-structured code with meaningful variable names and comments	Shorter code, fewer comments, more developer-style
Readability	Very beginner-friendly and easy to understand	Slightly less verbose but clean and professional

Task 3 — Leap Year using Cursor AI

Generate leap year program using **two different prompts**

The screenshot shows the Cursor AI interface with a dark theme. At the top, a chat window displays the prompt: "Write a Python program to check if a year is leap year." Below the prompt, the AI's response is shown as a code snippet in a file named "leap_year.py (new)". The code defines a function "is_leap_year(year)" with a docstring explaining the logic for leap years: "A year is a leap year if: - It is divisible by 4, AND - It is NOT divisible by 100, OR - It IS divisible by 400". The code also includes an "Args:" section for "year (int): The year to check" and a "Returns:" section for "bool: True if the year is a leap year, False otherwise". The code snippet ends with "if year % 400 == 0:". At the bottom of the interface, there is a status bar with "5 Files" and buttons for "Undo All", "Keep All", and "Review".

```

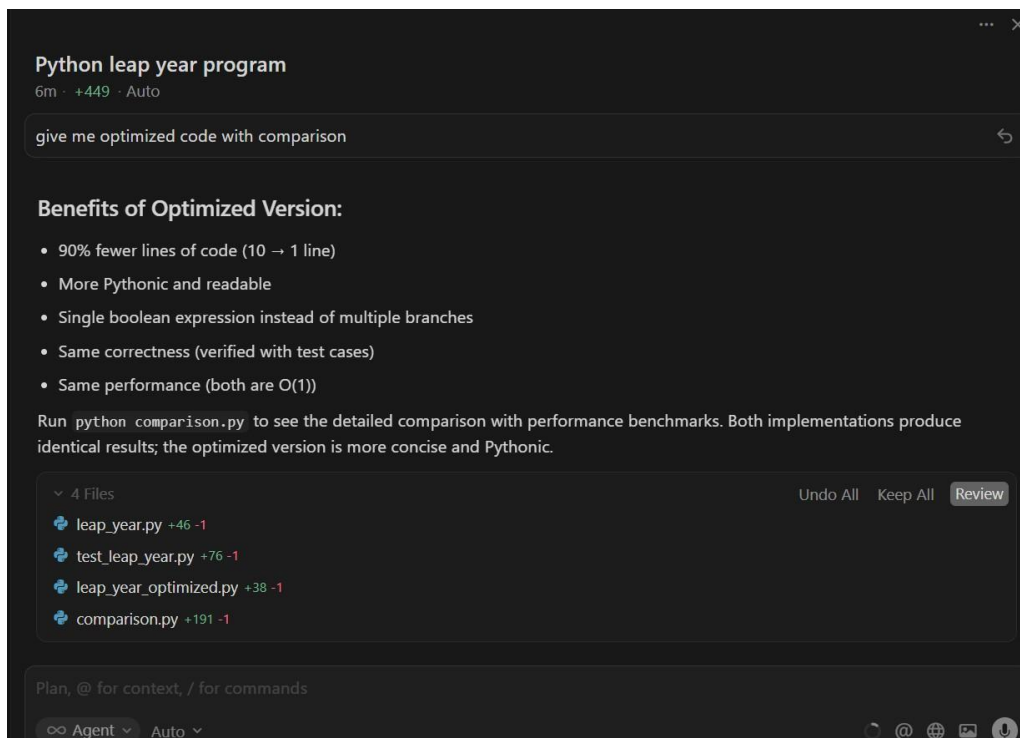
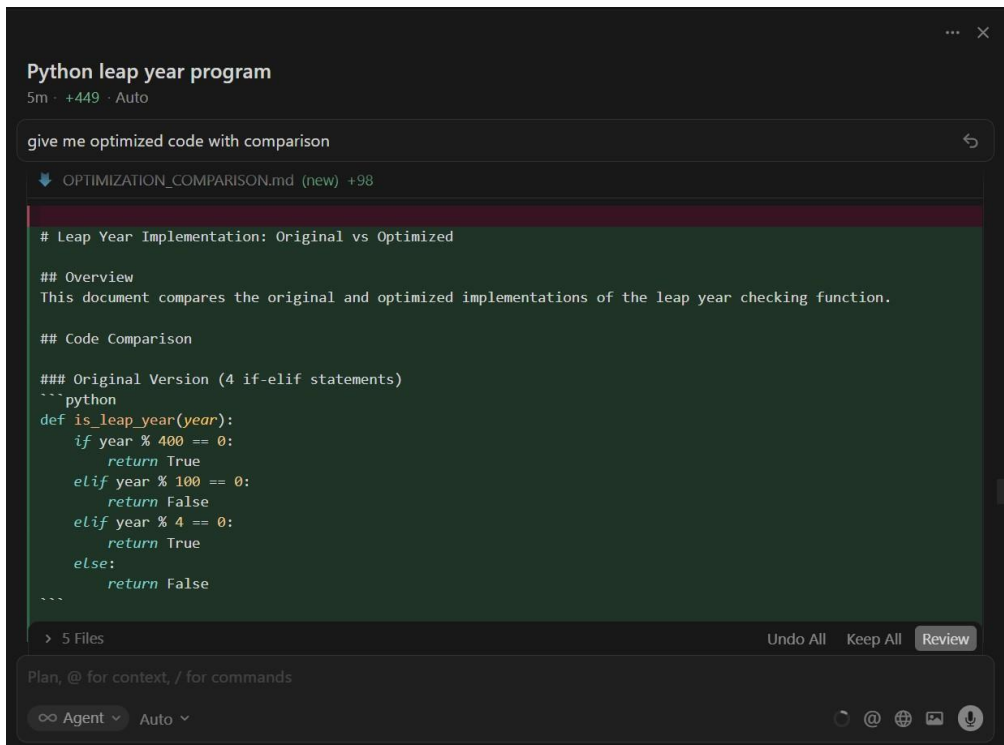
def is_leap_year(year):
    """
    Check if a given year is a leap year.

    A year is a leap year if:
    - It is divisible by 4, AND
    - It is NOT divisible by 100, OR
    - It IS divisible by 400

    Args:
        year (int): The year to check

    Returns:
        bool: True if the year is a leap year, False otherwise
    """
    if year % 400 == 0:

```



Task 4 — Odd/Even Sum (Manual + AI Refactor)

Manual Code

```

# Tuple of numbers
t = (10, 15, 20, 25, 30, 35)

even_sum = 0
odd_sum = 0

# Loop through each number
for num in t:
    if num % 2 == 0:
        even_sum += num
    else:
        odd_sum += num

print("Sum of Even Numbers:", even_sum)
print("Sum of Odd Numbers:", odd_sum)

```

Output:

```

Sum of Even Numbers: 60
Sum of Odd Numbers: 75

```

AI Refactored Code (Optimized Version)

```

def calculate_even_odd_sum(numbers):
    """Returns the sum of even and odd numbers from a tuple."""
    even_sum = sum(num for num in numbers if num % 2 == 0)
    odd_sum = sum(num for num in numbers if num % 2 != 0)
    return even_sum, odd_sum

numbers = (10, 15, 20, 25, 30, 35)
even, odd = calculate_even_odd_sum(numbers)

print(f"Sum of Even Numbers: {even}")
print(f"Sum of Odd Numbers: {odd}")

```

Output :

```

Sum of Even Numbers: 60
Sum of Odd Numbers: 75

```